Situational Awareness Dashboard

TypeScript · React · Performance · Testing

Duration	3–4 hours (no hard time limit)
Stack	React 18+ · TypeScript (strict mode) · your choice of CSS approach
Allowed	Any libraries, AI tools, and internet resources — we assess the result and your reasoning, not the method
Deliverable	Git repository (GitHub / GitLab / Bitbucket) with a README explaining setup and any design decisions

1 Overview

Build a situational awareness dashboard consisting of three widgets arranged as cards on a full page: a **Weather widget**, a **Delayed Routes list**, and a **Ramp Algorithm chart**.

Hardcoded data: The Weather widget, Delayed Routes list, and Network Summary panel are all driven by hardcoded data. Each should load its data from a local JSON file via a custom hook — do not inline the data directly in the component.

Live data: The Ramp Algorithm chart is the only widget that consumes live data. The chart and its history sparkline must reflect each update as it arrives.

■ AI-assisted development is permitted and expected at this level. We are evaluating your architectural decisions and ability to reason about the code — not whether you typed every character.

2 Ramp Data API (provided — do not modify)

The mock API below simulates a real-time data stream. It calls your callback with 50 ramp objects every 500 ms.

```
export type Algorithm =
| 'Algorithm 1' | 'Algorithm 2' | 'Algorithm 3'
| 'Algorithm 4' | 'Algorithm 5';

export interface Ramp {
id: string;
algorithm: Algorithm;
}

function getRampAlgorithms(onUpdate: (ramps: Ramp[]) => void): () => void {
const count = 50;
const id = setInterval(() => {
const ramps: Ramp[] = Array.from({ length: count }, (_, i) => ({
```

```
id: `ramp-${i}`,
algorithm: (['Algorithm 1','Algorithm 2','Algorithm 3',
'Algorithm 4','Algorithm 5'] as Algorithm[])[
Math.floor(Math.random() * 5)
],
}));
onUpdate(ramps);
}, 500);
return () => clearInterval(id); // returns a cleanup function
}

export default getRampAlgorithms;
```

3 Requirements

3b Functional requirements

Weather widget	Display current weather conditions including city, temperature, date and time, weather icon, and a breakdown of humidity, chance of rain, wind speed, and tomorrow's forecast. Weather data is hardcoded — no API call is required.
Delayed routes	Display a list of delayed routes. Each route shows a severity indicator, route name, via path, distance in km, and delay in minutes. Severity must be visually distinct between high and medium.
Ramp chart	The Ramp Chart is the only widget that displays live data. Display a donut chart showing the percentage of ramps currently running each algorithm. The chart must update in real time as new data arrives from the provided API.
History sparkline	Below the donut chart, show a line chart of the last 60 seconds of data for the dominant algorithm. This chart is called the **History Sparkline**. It tracks how the leading algorithm's share has shifted over time — for example, if Algorithm 3 currently holds 30% of ramps, the sparkline plots that value every 500 ms so the operator can see whether it is rising, falling, or stable.
Pause toggle	A pause/resume button at the bottom of the Ramp Chart must freeze the displayed chart and sparkline while data continues to be received in the background.
Network summary	Display a summary panel showing total ramps, active ramps, number of incidents, and average delay.

3c Architecture & state design

Component tree	Structure your components so that each widget is independently testable. Avoid prop-drilling more than one level deep — use composition, Context, or a lightweight state solution as appropriate. Document your choice in the README.
Hardcoded data	Weather and delayed route data may be hardcoded as typed constants. Define them in a `src/data/` directory with appropriate TypeScript types — not as untyped literals inlined in JSX.

3d Tests

Coverage expectation	Unit tests are expected for all code you write. Every custom hook, utility function, and non-trivial component should have accompanying tests. Reviewers will assess test coverage as part of code quality.

4 Evaluation Criteria

Area	What we look for
TypeScript	Type safety depth, discriminated unions, generics, no unsafe casts
React	Hook design, memoisation strategy, state architecture
UI fidelity	Visual accuracy against the mock, CSS quality, responsive behaviour
Testing	Test meaningfulness, fake timer usage, edge case coverage
Code quality	Readability, separation of concerns, README clarity

5 Tips & Reminders

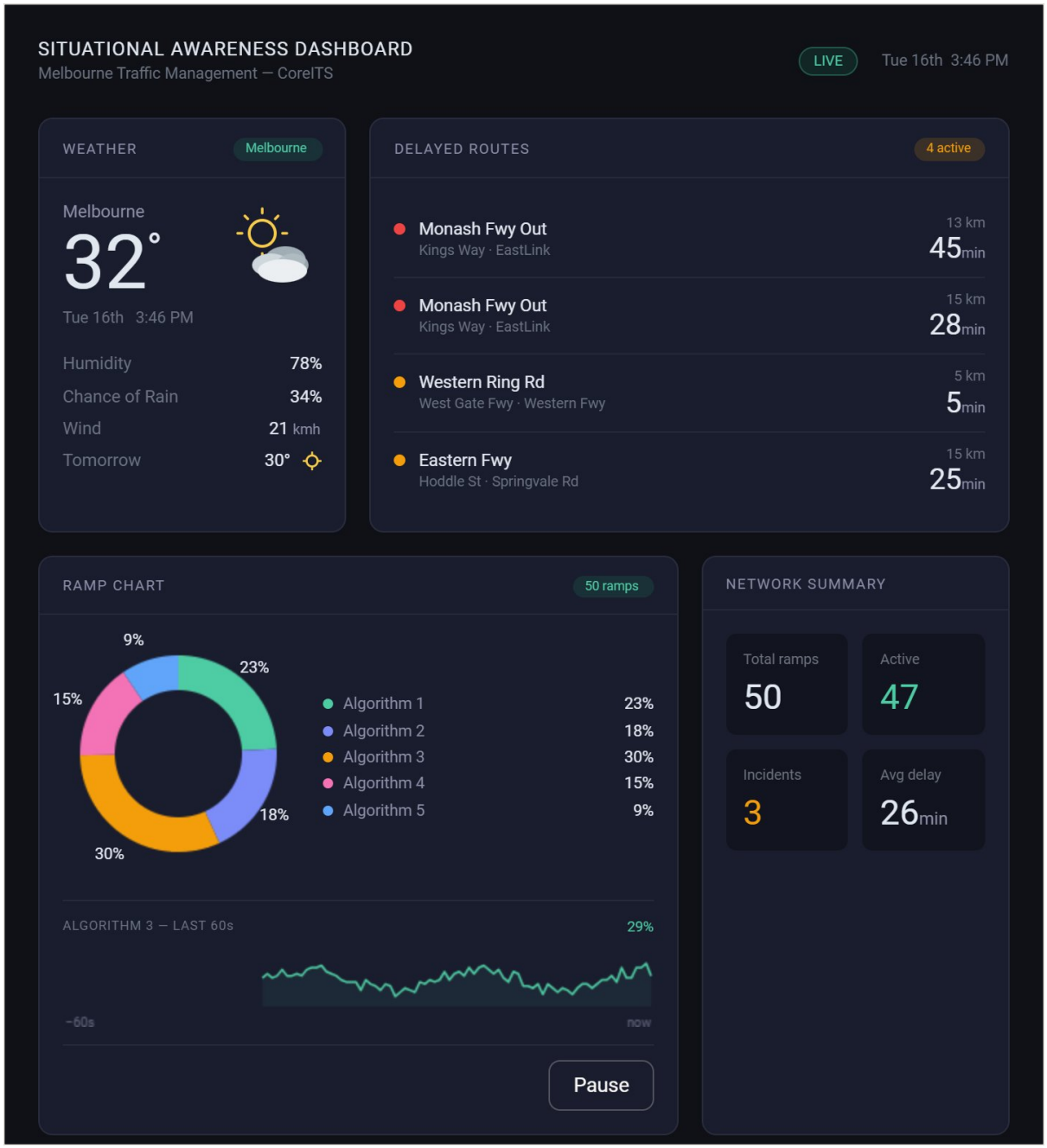
Roboto font	Use Google Fonts — add the import to your index.html or global CSS.
Donut chart	A pure SVG or Canvas donut chart is fine. A charting library (Recharts, Victory, Chart.js) is equally acceptable — pick whichever you are comfortable explaining.
README	A good README earns marks. Explain your state management decisions, memoisation strategy, and any tradeoffs you consciously made.

6 Submission

→ Push your work to a public or private Git repository and share the link. If private, please grant access before the interview. Include setup instructions in your README — the reviewer should be able to run the project with two commands.

Confidential — for candidate use only. Please do not share or publish this assessment.

Dashboard — UI Mock

Your implementation should closely replicate this layout, colour scheme, and typography.

Element	Notes
Weather widget	City name, large temperature, date/time, weather icon, and four metric rows (Humidity, Chance of Rain, Wind, Tomorrow). Data is hardcoded.
Delayed Routes	Each route: severity dot (red=high, amber=medium), route name, via path, distance and delay in minutes.
Ramp Chart	Donut chart with legend, 60-second history sparkline, and a pause/resume button at the bottom.
Network Summary	Four stat cards: Total ramps, Active, Incidents, and Avg delay.
Severity typing	Model severity as a discriminated union — 'low' | 'medium' | 'high' — mapped to CSS classes via mapped type or satisfies.